Supplementary Material for FINJECT: Expanding Finance Reasoning Problems through Injection of Unanswerability

Supplementary Material

This supplementary document contains construction, judging, annotation, and diagnostic details that support the main four-page CIKM 2026 Resource Track submission. The main paper is self-contained; these materials are provided for reproducibility and auditability.

1 Deterministic Validation (Stage 1)

Stage 1 is a deterministic validation layer, not an LLM judge. It converts the global perturbation instructions and the category-specific constraints into executable checks over each generated candidate. The purpose is to remove malformed candidates before semantic judging.

At a high level, the conformance Score is built from two parts:

- **Gates** (credits): added when the required transformation properties hold.
- **Penalties**: subtracted when forbidden patterns appear.

Algorithm 1 shows the validation flow: a malformed-candidate guard, then the credit (gate) and penalty (deduction) checks, the score of Eq. (1), and the PASS decision.

Algorithm 1 Stage 1 deterministic validation for one candidate.

Require: category cat, original context c_0, perturbed context c_1, reference solution sol
Ensure: (PASS/FAIL, Score)

if c_1 is empty or the schema is invalid **then**
  return (FAIL, 0) ▷ malformed candidate
end if
$\mathcal{G} \leftarrow \textsc{GlobalGates} \cup \textsc{CategoryGates}(cat)$ ▷ required positive checks (credit)
$\mathcal{P} \leftarrow \textsc{Penalties}(cat)$ ▷ forbidden-pattern checks (deduction)
Score $\leftarrow$ combine $\mathcal{G}$ and $\mathcal{P}$ via Eq. (1) ▷ gate credits − penalties, clamped to $[0, 100]$
PASS $\leftarrow$ **true** iff every applicable gate in $\mathcal{G}$ holds ▷ penalties do not affect PASS
return (PASS, Score)

The Score is defined as:

$$\text{Score} = \max\Big(0,\ \min\big(100,\ \underbrace{S_{\text{global}}}_{\text{system-prompt gates}} \cdot\alpha\ +\ \underbrace{S_{\text{category}}}_{\text{type gates}} \cdot(1-\alpha)\ -\ \underbrace{\textstyle\sum_k P_k}_{\text{penalties}}\ \big)\Big). \tag{1}$$

Here $\alpha = 0.2$; S_{global} and S_{category} are the weighted pass rates of the *applicable* global and category gates (a type with no gate, e.g. EA-full, scores S_{category}=100), and each P_k is a deduction triggered by a forbidden pattern. PASS/FAIL is decided by the gates alone; the penalties lower the score but never flip the decision.

The gates and penalties are taken directly from the perturbation prompts (system prompt §3.1; per-category instructions §3.3). Table S1 lists the gates (credits) — global, behind S_{global}, and per-category, behind S_{category} — and Table S2 the per-category penalties P_k; each row pairs the prompt clause with its score.

Table S1: Gates (credits) behind S_{global} and S_{category}. The two global gates (system prompt, §3.1) apply to every category — the second (value preservation) only to the conflict types IC-value/IC-source. Category gates (§3.3) are type-specific; their per-type weights sum to 100.

Category	Prompt clause (§3.1 / §3.3)	Score
Global	"Do NOT change the question; transform the context only." (§3.1)	+50
	"For conflict types, ADD a contradicting value or premise rather than removing data." (§3.1)	+50
EA-partial	"replace ONLY that value with the explicit marker `[DATA MISSING]` ... Replace exactly ONE essential value" (§3.3)	+100
EA-full	"DELETE the entire sentence/clause that carries the essential value" (§3.3)	—†
SA	"Remove ONLY the numeric VALUE of the essential quantity" (§3.3)	+100
IC-value	"Modify the context so BOTH values appear ... add a second natural statement of the same quantity" (§3.3)	+100
IC-source	"ADD a conflicting value (1.3x – 1.7x of the original)"	+50
	"add ... 'The two figures have not been reconciled ...'" (§3.3)	+50
IC-premise	"assert TWO statements that CANNOT both be true" (§3.3)	+50
	"ADD ... rather than removing data" (§3.1)	+50

† EA-full defines no category gate, so S_{category}=100 by default; the deletion requirement is instead enforced by the essential-not-deleted penalty (−40, Table S2).

Table S2: Penalties P_k: forbidden-pattern clauses and their deductions, by category. The two global penalties apply to every category (marker-added to all types except EA-partial). String patterns (attributions, cues, markers, trace words) are detected by regular-expression matching.

Category	Prompt clause (§3.1 / §3.3)	Score
Global	"no semicolons, em-dashes, or other markers absent from the original" (§3.1)	−20
	"Leave NO marker" / "NEVER leave a marker" (§3.3; all but EA-partial)	−20
EA-partial	"Majority unchanged" (§3.1/§3.3)	−20
	"the explicit marker `[DATA MISSING]`" (§3.3)	−20
EA-full	"DELETE the entire sentence/clause ... must contain data essential" (§3.3)	−40
	"add a neutral sentence ONLY if needed" (§3.3)	−20
	no new / back-derivable data (§3.1, §3.3)	−20
SA	"Remove exactly one essential ... value" (§3.3)	−20
	"A reader must NOT feel a value is missing" (§3.3)	−20
	keep the rest natural; no fabricated data (§3.3)	−20
IC-value	"A digit-order(x10) or absurd value is FORBIDDEN" (§3.3)	−20
	"FORBIDDEN: attribution phrases" (§3.3)	−20
	"Provide NO cue (no 'corrected'/'updated' ...)" (§3.3)	−20
IC-source	"never join with a semicolon" (separate reconcile sentence) (§3.3)	−20
	"NEVER mark one source as authoritative / audited ... UNLESS ... both" (§3.3)	−10
	"Provide NO recency / version / correction clue." (§3.3)	−10
IC-premise	"do NOT attribute either premise to ... authority" (§3.3)	−20
	"No cue may indicate which statement is the error." (§3.3)	−20

The exact implementation is released with the dataset construction code. The main paper reports Stage 1 as a filter over 1,872 generated candidates; 1,690 candidates pass this deterministic gate before Stage 2 semantic validation when malformed, empty, or unevaluated cells are counted as structural failures. Table S3 shows the four candidates that explain the difference from the earlier evaluated-cell export. All four were already rejected by Stage 2, so the validated candidate pool, release sample, human verification statistics, and downstream evaluation rows are unchanged.

Table S3: Stage 1 accounting audit for the four candidates that differ from the earlier evaluated-cell export.

Generator	Category	Source qid	Stage 2 votes
Gemini 2.5 Pro	EA-partial	test-2049	0/1
Gemini 2.5 Pro	EA-partial	test-2207	0/1
Gemini 2.5 Pro	SA	test-2049	0/1
R1-Distill-32B	EA-full	test-2036	0/1

Each row was previously counted in a Stage 1 evaluated-cell export but is counted as a structural failure under the final all-candidate denominator because the transform was malformed, empty, or unevaluated. Since each row also has zero Stage 2 unanswerability support among parsed votes, none can satisfy FinalPass.

2 Semantic Judge Protocol (Stage 2)

Stage 2 checks semantic unanswerability after Stage 1 removes malformed candidates. Each candidate is judged only by model families different from the generator family. The judge sees the original problem, the perturbed problem, the intended perturbation category, and the category rule. It does not receive human labels, release sampling decisions, downstream results, or generator self-judgments.

Each judge returns two binary labels:

- **Category validity**: whether the perturbation still matches the intended category.
- **Unanswerability**: whether the perturbed problem lacks a uniquely supported answer.

The main automatic pass rule uses non-self majority over unanswerability labels:

$$\text{Stage2Pass}(x) = \mathbb{1}\left[\sum_{j \in J_x} \mathbb{1}(\text{Unanswerable}_j(x) = \text{PASS}) \geq 2\right].$$

Category-validity votes are retained for strict sensitivity analysis. This separation avoids double-counting structural category validity because Stage 1 already applies deterministic category checks.

Table S4: Stage 2 judge output schema.

Field	Meaning
`category_valid`	PASS/FAIL category-semantics label
`unanswerable`	PASS/FAIL semantic unanswerability label
`reason`	short judge rationale
`parse_ok`	whether the judge output matches the schema
`judge_model`	non-self judge family

2.1 Semantic Judge Prompt

```
You are validating whether a perturbation of a financial reasoning problem is a
GENUINE unanswerable/underdetermined instance.

Inputs: ORIGINAL question+context, TRANSFORMED context, INTENDED type
(one of: EA-partial, EA-full, SA, IC-value, IC-source, IC-premise).

Judge two INDEPENDENT axes (do not let one bias the other):

[Axis 1 -- Category validity]
Does the transform match the intended type's definition?

[Axis 2 -- Unanswerability]
```

```
Judge ONLY whether the answer is uniquely determinable.
- If ABSENCE type: PASS iff the essential quantity is gone AND cannot be
  back-derived from any remaining data.
- If CONFLICT type: PASS iff ALL hold:
  (a) both conflicting values/premises are individually plausible;
  (b) NO in-context cue (authority, recency, correction, definitional identity,
      third reconciling value, or plausibility gap) lets a solver prefer one side;
  (c) at least two different valid answers exist depending on the side chosen.

Also report which side a typical model would pick and why; this exposes hidden
preference cues.

Output ONLY JSON:
{
  "category_valid": "PASS|FAIL",
  "unanswerable": "PASS|FAIL",
  "plausibility_ok": true,
  "preference_cue": "<cue or 'none'>",
  "back_derivable": false,
  "two_valid_completions": true,
  "likely_model_choice": "<value/premise or 'abstain'>",
  "reason": "<2-3 sentences>"
}
```

3 Perturbation Generation Prompt

All generator families use the same task framing and output schema. The model receives the original question, original context, correct answer, and executable reference solution. The reference solution is used only for construction: it identifies which values, premises, or source choices are *answer-critical*, meaning necessary to derive the unique numeric answer. In the prompt text below, `load-bearing` is used as a synonym for this answer-critical evidence.

3.1 Shared System Prompt

```
You are a financial-data perturbation expert.

You will be provided with a QUESTION, a CONTEXT, the CORRECT ANSWER, and a
reference SOLUTION. The answer is currently DEDUCIBLE from the context. Your
task is to modify ONLY the context, per the given Transformation Type, so that
a competent solver can no longer derive a unique correct answer. This evaluates
whether LLMs can recognize unanswerable or underdetermined problems
(metacognition / abstention).

Method (all types):
1. Use the reference solution to identify the data ESSENTIAL to the answer.
2. Review the context SENTENCE BY SENTENCE; keep the MAJORITY of it unchanged
   and touch only what the Transformation Type requires.
3. The result must read as fluent, grammatically complete, and natural English.
4. Do NOT change the question; transform the context only. If there is no
   context, transform the question itself.
5. For conflict types, ADD a contradicting value or premise rather than
   removing data.
6. Output ONLY the specified JSON object -- no commentary, no code fences --
   and fill "derivable_check" with a justification of why the answer is now
```

```
   underivable or underdetermined.
```

3.2 Generator Output Schema

```
{
  "is_transformable": true,
  "reason_if_not": null,
  "critical_data": ["<solution-essential data>"],
  "removed_or_modified": "<what was removed/altered, or the conflict added>",
  "derivable_check": "<why no unique answer is now derivable>",
  "transformed_context": "<full transformed context>",
  "description": "<one-line summary>"
}
```

3.3 Category-Specific Instructions

EA-partial.

```
EA-partial -- Explicit Absence (Partial).

In the sentence that states the essential value, replace ONLY that value with
the explicit marker "[DATA MISSING]" (keep the rest of the sentence intact).
- Replace exactly ONE essential value, and only one that is NOT back-derivable
  from the remaining data (if Total = A + B, do NOT mark A -- Total - B
  recovers it).
- The marker must be visible; the reader should clearly see a value is missing.
- Sentences that do not contain the essential value: leave unchanged.
- derivable_check: justify the marked quantity cannot be recovered by ANY
  alternate path from the remaining data.
```

EA-full.

```
EA-full -- Explicit Absence (Full).

DELETE the entire sentence/clause that carries the essential value (and only
that). Leave NO marker. You may add a neutral sentence ONLY if needed for
coherence.
- The deleted block must contain data essential and not back-derivable.
- Remaining text must be grammatically complete; majority unchanged.
- For tables or lists, target the load-bearing row, cell, or entry rather than
  deleting the whole table or data series.
- derivable_check: justify the quantity is gone and not recoverable from the
  rest.
```

SA.

```
SA -- Silent Absence.

Remove ONLY the numeric VALUE of the essential quantity, while KEEPING a natural
reference to the quantity itself; rewrite the sentence so it reads complete.
- The quantity must still be MENTIONED but its number is silently gone.
  Deleting the quantity's mention entirely is EA-full, NOT SA.
- NEVER leave a marker ("N/A", "[DATA MISSING]").
- Remove exactly one essential, non-back-derivable value; keep the rest natural.
- A reader must NOT feel a value is missing -- naturalness is the key criterion.
```

```
- For tables, preserve the table and remove or neutralize only the load-bearing
  numeric cell when possible.
- derivable_check: justify (1) the number is absent and unrecoverable, and
  (2) the quantity is still referenced with no explicit trace.
```

IC-value.

```
IC-value -- Intra-context Value Conflict.

Pick ONE essential numeric value. State a SECOND value for the SAME quantity
that is DIFFERENT but FINANCIALLY PLAUSIBLE: same order of magnitude / within
the valid domain range (e.g., tax rate 25% -> 28%; margin 12% -> 15%). A
digit-order or absurd value is FORBIDDEN.
- Modify the context so BOTH values appear.
- Keep the original statement and add a second natural statement elsewhere.
- Place the second value in a different, non-adjacent sentence when possible.
- Do NOT attribute the two values to different sources, and do NOT use different
  units; those are other conflict types.
- Provide no cue such as "corrected", "updated", "approximately", or a third
  reconciling value.
- derivable_check: justify both values are individually plausible and no signal
  lets a solver prefer one.
```

IC-source.

```
IC-source -- Inter-source Value Conflict.

Keep the original value. ADD a conflicting value for the SAME essential
quantity, and attribute the TWO values to two financial sources of EQUAL
authority, EQUAL recency, and EQUAL TOPICAL RELEVANCE.
- Both values must be financially plausible and realistic.
- Avoid pairs where one source is more naturally tied to the quantity.
- NEVER mark one source as authoritative, official, audited, latest, revised,
  corrected, final, system of record, management confirmed, or equivalent,
  unless the same marker applies equally to both.
- Provide NO recency / version / correction clue.
- Add a separate sentence stating that the two figures have not been reconciled
  and neither source is designated as authoritative.
- derivable_check: justify there is no principled basis to prefer either source.
```

IC-premise.

```
IC-premise -- Mutually-Exclusive Premise.

Insert or alter the context to assert TWO statements that CANNOT both be true,
where BOTH are load-bearing for the question.
- The conflict must be logical or categorical, NOT a numeric mismatch of the
  same quantity; numeric mismatch is IC-value.
- The conflict must be factual, independently reasonable, and natural in the
  original context.
- Use distinct wording and explicit entity names.
- No cue may indicate which premise is the error.
- Prefer conflicts over accounting rule, inclusion criterion, contractual
  condition, payoff rule, tax treatment, amortization rule, window definition,
  or temporal applicability.
```

```
- Do not create IC-premise by merely stating two different values for the same
  quantity, including implicit zero-vs-positive forms.
- derivable_check: justify the premises are logically incompatible, individually
  plausible, and neither can be dismissed.
```

3.4 Human-Calibrated Re-perturbation

Candidates rejected by both human annotators are not replaced by arbitrary Stage 1/Stage 2 alternatives. They are regenerated by the same original generator family when possible, using the original problem, the failed transformed context, and the human feedback. This preserves the multi-generator construction design while correcting the specific failure mode observed during annotation.

```
You are retrying a FInject perturbation that failed human annotation. The
previous perturbation below must NOT be copied. Read the human feedback and
create a NEW transformed context for the same original problem and same
category.

Additional human-calibrated retry rule:
Keep the original question and the context structure. Replace exactly one
load-bearing numeric value (or, for conflict types, add exactly one conflicting
value/premise). For tables or lists, edit one load-bearing cell/entry only; do
not rewrite the whole table into prose and do not delete the question/action
clause.

Output ONLY this JSON object:
{
  "is_transformable": true,
  "reason_if_not": null,
  "critical_data": ["<essential data used by the solution>"],
  "removed_or_modified": "<specific value/premise/source changed>",
  "derivable_check": "<why the answer is no longer uniquely derivable,
                       explicitly addressing the human feedback>",
  "transformed_context": "<full new transformed context only>",
  "description": "<one sentence summary>",
  "human_feedback_addressed": ["<how the retry fixes each objection>"]
}
```

For R1-Distill-32B rows that required external inference, we exported a handoff packet containing one JSONL task per failed row. Each task included the original question, original context, reference solution, previous failed perturbation, annotator feedback, intended category, category instruction, and retry rule. The worker returned one JSONL row per task while preserving `retry_id`, `source_qid`, `category`, and `generator_model`, so the regenerated contexts could be merged back into the final 426-instance release.

4 Answerability Evaluation Prompt

The same evidence-sufficiency prompt is used for original controls and perturbed negatives. The model receives only the question and context. The full template is:

```
You are solving a financial numerical reasoning problem.

Use only the provided question and context. Do not use outside knowledge,
unstated assumptions, memorized benchmark answers, or plausible default values.
Do not call external tools, calculators, code execution, web search, or files.
```

```
Follow this procedure internally before producing the final JSON:
1. Identify the formula or reasoning procedure needed to answer the question.
2. Identify every quantity, premise, condition, time period, and source choice
   required by that procedure.
3. Check whether each required item is explicitly available in the context.
4. Check whether any required item has unresolved conflicting values, source
   statements, or premises. If two plausible alternatives are present and the
   context gives no explicit priority rule, the answer is not uniquely supported.
5. If any required item is missing, not recoverable from the context, or
   contradicted without a priority rule, do not guess.
6. If the context is sufficient and non-contradictory, compute the answer
   carefully using step-by-step arithmetic internally.

Return only valid JSON in one of the following two forms:

{"decision":"INSUFFICIENT_INFORMATION","answer":null}
```

or:

```
{"decision":"ANSWER","answer":<numeric answer>}

The numeric answer should follow the rounding and unit instructions in the
question. Do not include explanations, code fences, or extra keys.

Question:
{question}

Context:
{context}
```

The numeric answer must follow the rounding and unit instructions in the question. Non-JSON outputs are retried once during evaluation; remaining negative-context parse failures are reported as `ParseFail`.

5 Answerability Metric Formulas

Let O be the set of answerable original controls and N be the set of final-release negative instances. Original accuracy is:

$$\text{OrigAcc} = \frac{1}{|O|} \sum_{o \in O} \mathbb{1}[\text{CorrectAnswer}(o)].$$

Refusal precision treats correct refusals on negatives as true positives and over-refusals on originals as false positives. Refusal recall is the fraction of negatives refused:

$$P_{\text{ref}} = \frac{\sum_{n \in N} \mathbb{1}[\text{Refuse}(n)]}{\sum_{n \in N} \mathbb{1}[\text{Refuse}(n)] + \sum_{o \in O} \mathbb{1}[\text{Refuse}(o)]},$$
$$R_{\text{ref}} = \frac{\sum_{n \in N} \mathbb{1}[\text{Refuse}(n)]}{|N|},$$
$$F1_{\text{ref}} = \frac{2 P_{\text{ref}} R_{\text{ref}}}{P_{\text{ref}} + R_{\text{ref}}}.$$

The hallucination rate is:

$$\text{HallucinationRate} = \frac{\sum_{n \in N} \mathbb{1}[\text{Answer}(n)]}{|N|}.$$

Paired success requires a correct original-control answer and refusal on the paired negative:

$$\text{PairedSuccess}(x) = \mathbb{1}[\text{CorrectOriginal}(x) \wedge \text{RefuseNegative}(x)].$$

Table S5: Detailed FinalPass matrix by generator and category.

Generator	Macro	Strict	Min cat.	Pass	EA-partial	EA-full	SA	IC-value	IC-source	IC-premise
Claude Opus	90.6	88.9	79.5	424/468	97.4 (76/78)	94.9 (74/78)	97.4 (76/78)	93.6 (73/78)	79.5 (62/78)	80.8 (63/78)
Gemini 2.5 Pro	80.1	79.1	55.1	375/468	96.2 (75/78)	96.2 (75/78)	94.9 (74/78)	78.2 (61/78)	60.3 (47/78)	55.1 (43/78)
GPT-5.5	74.6	69.4	30.8	349/468	93.6 (73/78)	84.6 (66/78)	93.6 (73/78)	85.9 (67/78)	30.8 (24/78)	59.0 (46/78)
R1-Distill-32B	66.9	59.6	19.2	313/468	94.9 (74/78)	97.4 (76/78)	97.4 (76/78)	52.6 (41/78)	39.7 (31/78)	19.2 (15/78)

We report the average paired success rate over N and summarize abstention quality as:

$$\text{MCScore} = \text{RefusalF1} \times (1 - \text{HallucinationRate}).$$

Repeated non-JSON or empty outputs are counted as incorrect on originals and as non-refusals on negatives. Negative parse failures are reported separately.

6 Human Annotation Summary

Human verification labels each sampled release candidate on two independent axes. The first axis, *category validity*, checks whether the perturbation implements the intended category. The second axis, *unanswerability*, checks whether the transformed problem lacks a uniquely supported answer under the supplied context.

Labels are binary PASS/FAIL. Ambiguous cases are marked FAIL with a reason code and a short note. Reason codes are grouped by axis:

- `cv_*`: category-validity failures, e.g., over-deletion, marker-format errors, wrong conflict type, or deletion artifacts.
- `ua_*`: unanswerability failures, e.g., same value elsewhere, back-derivation, action anchor, source priority, or premise reconciliation.

The annotation interface hides automatic judge labels and generator identity. It shows the original problem, transformed problem, intended category, and executable reference solution so annotators can determine which evidence is answer-critical. The released benchmark retains row-level provenance for repaired candidates.

7 Detailed Experiment Tables

Table S5 expands the construction summary in the main paper by reporting every perturbation category for each generator family. The release is not selected from the best single row; it is sampled from the validated multi-generator union and then repaired through human verification.

Table S6 reports the full two-axis human verification statistics. The compact main-paper table reports only the derived inclusion label, while this table exposes the category-validity and unanswerability axes separately.

Table S7 reports the full answerability scoring output for the completed evaluation rows. The main paper removes intermediate precision, recall, and parse-failure columns to keep the central result readable.

Table S8 gives representative final-release negative instances where frontier models returned numeric answers despite a visible evidence gap. Each case reports the model's observed output and the specific missing or conflicting evidence that makes the numeric answer unsupported.

8 Additional Diagnostics

Table S9 is a diagnostic creator self-evaluation. It is not the main non-self answerability benchmark table in the paper: each model is evaluated only on its own generated negatives, so the rates should not be compared directly to the non-self benchmark rows.

Table S6: Detailed human verification by category and axis.

Category	N	Joint fail	Release	CV agr./κ/dis.	UA agr./κ/dis.	Final agr./κ/dis.
EA-partial	71	11	71	100.00 / 1.000 / 0	95.77 / 0.386 / 3	95.77 / 0.855 / 3
EA-full	71	15	71	95.77 / 0.882 / 3	95.77 / 0.000 / 3	91.55 / 0.777 / 6
SA	71	17	71	97.18 / 0.926 / 2	94.37 / 0.000 / 4	91.55 / 0.791 / 6
IC-value	71	1	71	100.00 / 1.000 / 0	95.77 / 0.386 / 3	95.77 / 0.386 / 3
IC-source	71	19	71	100.00 / 1.000 / 0	90.14 / 0.773 / 7	90.14 / 0.773 / 7
IC-premise	71	7	71	98.59 / 0.915 / 1	98.59 / 0.850 / 1	97.18 / 0.859 / 2
Overall	426	70	426	98.59 / 0.934 / 6	95.07 / 0.670 / 21	93.66 / 0.799 / 27

Low UA κ for EA-full and SA reflects one-sided label skew: both annotators almost always agree that the transformed problem is unanswerable, so a few disagreements yield $\kappa = 0$ despite high raw agreement. The final release repairs all 70 joint failures and retains row-level provenance.

Table S7: Detailed answerability evaluation metrics.

Model	Group	N_{orig}	N_{neg}	Orig. Acc.	Ref. P	Ref. R	Ref. F1	PairSucc	Halluc.	ParseFail	MCScore
GPT-5.5	gen-family	78	320	89.7	97.5	87.2	92.1	76.9	12.5	1	80.6
Claude Opus 4.7	gen-family	78	318	85.9	99.2	81.5	89.5	68.2	18.6	0	72.9
Gemini 2.5 Pro	gen-family	78	318	82.0	99.3	84.0	91.0	69.2	16.0	0	76.4
R1-Distill-32B	gen-family	78	322	66.7	99.0	64.3	78.0	45.0	35.1	2	50.6
Llama-3.3-70B	disjoint open	78	426	25.6	93.5	81.2	86.9	18.5	18.8	0	70.6
Qwen2.5-72B	disjoint open	78	426	16.7	98.5	78.4	87.3	12.2	21.4	0	68.7
DeepSeek V3.1	disjoint open	78	426	28.2	98.2	75.8	85.6	20.9	24.2	0	64.9
Qwen3-8B	disjoint open	78	426	94.9	99.7	67.8	80.7	64.6	32.2	0	54.8
Mistral Large	disjoint API	78	426	34.6	97.3	85.2	90.9	31.0	14.8	0	77.4
Cohere Command A	disjoint API	78	426	7.7	94.2	84.3	89.0	5.9	15.5	0	75.2
Grok 4.20	disjoint API	78	426	15.4	96.8	91.8	94.2	12.9	8.2	0	86.5

Table S8: Representative frontier hallucination cases.

Evaluator	Category / source	Perturbed evidence gap	Observed output	Why this is a hallucination
GPT-5.5	IC-value / test-2089	Yield to maturity appears as both 4.5% and 4%, with no recency or authority cue.	`ANSWER`: 10445	The model chooses one discount rate and reports a present value.
Claude Opus 4.7	SA / test-2009	The Japanese borrowing rate is only described as "extremely low"; the Brazilian investment rate is 5%.	`ANSWER`: 2500	The model ignores the missing borrowing cost and computes a determinate profit.
Gemini 2.5 Pro	IC-premise / test-2048	The mortgage is both interest-only with a balloon principal payment and fully repaid over 30 years.	`ANSWER`: 900	The model answers under one payment rule despite incompatible premises.

Table S9: Creator self-evaluation on each generator family's final rows.

Model	Pairs	Orig. Acc.	Self Ref.	Self Halluc.	PairSucc
Claude Opus 4.7	108	89.8	76.9	23.1	68.5
Gemini 2.5 Pro	108	77.8	88.9	10.2	70.4
GPT-5.5	106	86.8	85.9	14.2	72.6
R1-Distill-32B	104	92.3	73.1	26.9	67.3